AI ENGINEERING COHORT · DELIVERABLE 4

Data Flow

Conversational Memory Intelligence System — Field-Level Transformations

Abhijeet Hiwale

August 2026

design/data_flow.pdf

1. Write Flow — Field-Level

Expands architecture.pdf's write-flow boxes and system_design.pdf Section 7's data model into what actually changes at each stage.

Conversation Turn (raw input)

```
{ text, user_id, tenant_id, timestamp, turn_type: user | assistant }
```

↓

Admission: classifies the operation, tags provenance, runs the PII pre-guardrail (system_design.pdf, Section 5 & 11)

Admission output

```
{ admission_op: ADD | UPDATE | DELETE | NOOP,
  provenance: user_stated | assistant_generated | tool_derived | retrieved_document,
  pii_scan_result: pass | flag | redacted,
  confidence }
```

↓

Storage: persists the full typed record (system_design.pdf, Section 7)

Stored MemoryRecord

```
{ id, tenant_id, text,
  dense_embedding, sparse_terms,
  provenance, admission_op,
  valid_from: now, valid_until: null,
  confidence, pii_scan_result,
  status: active,
  consolidation_lineage: [] }
```

If `admission_op` is UPDATE and a same-entity/slot record already exists, that prior record's `valid_until` is set (Week 1's supersession mechanism) — unless `confidence` was low at write time, in which case both records remain `status: active` and are marked conflicting rather than one being silently closed (Week 1 self-review fallback, system_design.pdf Section 9).

2. Read Flow — Field-Level

Expands the ranking formula from system_design.pdf, Section 9.

Query (raw input)

```
{ query_text, tenant_id, token_budget }
```

↓

Retrieval: tenant pre-filter (deterministic, before scoring) → BM25 + dense → RRF fusion → relevance floor

Retrieval output

```
{ candidates: [ { id, text, rrf_rank_score }, ... ] }
  — or an explicit empty result if nothing clears the relevance floor
```

↓

Ranking: supersession check, then final_score = w_r·rrf_rank_score + w_t·recency_weight(valid_from) + w_p·provenance_trust(provenance)

Ranking output

```
{ ranked_list: [ { id, text, final_score, superseded: bool }, ... ] }
```

↓

Context Builder: injects in ranked order into per-zone token budgets until each zone's ceiling is hit

Context Builder output (returned to caller)

```
{ injected_context: text,
  tokens_used, zones: { system_prompt, retrieved_memory, history, tool_output, input,
output_reserve } }
  — or { no_relevant_memory: true } as a first-class, non-error result
```

3. Background Flow — Field-Level

Expands the Lifecycle Worker box from architecture.pdf into the four-lever framework's actual state transitions (system_design.pdf, Section 7).

Storage scan (scheduled, not request-triggered)

```
Reads records where status = active,
  filtered by: decay-eligible (age vs. importance) OR provenance = unverified/low-confidence
  (Week 4's default-expiry rule)
```

↓

Lifecycle Worker applies one of three levers per eligible record:

Decay (soft — rank-affecting, not deletion)

```
status stays "active"; recency_weight used in Section 9's ranking formula
  decreases → record becomes less likely to surface, is never removed by this lever alone
```

Merge

```
new consolidated record created: { ..., consolidation_lineage: [source_id_1, source_id_2, ...] }
  source records: status → "merged" (retained, not deleted, for lineage integrity)
```

Eviction (hard — the only lever that touches deletion)

```
Triggered by: decay reaching a floor, OR an explicit deletion request.
  status → "evicted" | "deleted"
  On explicit deletion: walk consolidation_lineage backward from every derived record
  → re-consolidate or evict each one too (Week 4's "backflow" prevention requirement)
```

The eviction lever's lineage-walk step is this design's least-proven guarantee (system_design.pdf, Section 16) — the data flow above shows what it is supposed to do; Deliverable 6 is where it gets verified against a real implementation.